SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING) Department of Artificial Intelligence and Data Science CO1-ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Cour	DSA0311	**Cou**	Natural Language Processing
C	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – ApplicationBase Questions
Weightage:	40% of C		
CO1 State	Analyse NLP models, regular expression patterns, finite state automata, morphologicalprocessing techniques and to interpret language processing. (BL-3)		
Student Name:	U.Ananth Krishna	**Reg. No.:**	192425357
Section	**/ Batch:** AIML	**Date:**	10-10-2026

Code of Conduct

I U.Ananth Krishna/ 192425357 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student : U.Ananth Krishna

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirementsspecified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. TheHRdepartment requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction systemusing Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.

Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience andPythonskill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experiencethrough flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressionsthat performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search. **Section 3:**

Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information beforecourseenrollment.

Design and implement a Python-based validation system using Regular Expressions that performsthefollowing tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4Exemplary	
Understanding ProblemConte	20	Clearly understandst heproblem accurately identifies requirements	

Application Concepts		Accurately applies appropriate conceptsto complex problems
Problem-Solving Approach	20	Logical, well-structured, efficient approach
Accuracy Solution	20	Completely correctsoluti with justification
Clarity	10	Clear, well-organized, andproperly

		explained solution

Q. No.Crite a	/ Understanding Problem Context	Application of Concepts	Problem Solving Approac
1			
2			
3			

Faculty In-charge	Course Coor
Signature:	Signature:
Date:	Date:

NLP ASSESSMENT-1

Application-Based Questions

Using Python Regular Expressions Resume Information Extraction

• Product Search • University Registration Validation

Introduction

Regular Expressions (Regex) are patterns used to search, extract, and validate text. Python provides the remodule for operations such as search(), findall(), fullmatch(), and IGNORECASE. This document presents three application-based solutions using Regex.

Note: The interface images in this document are illustrative website-style mockups for presentation. Theyarenot screenshots of a deployed or running website. The Python programs below are the implementations.

Objectives

• Extract useful information from resume text.
• Search a product catalogue using exact, prefix, suffix, partial, and case-insensitive matching. • Validate student registration fields and report whether the registration is successful.

Section 1: Resume Information Extraction

1. Problem Statement

A recruitment company receives resumes in text format. The system must extract the candidate's name, email address, mobile number, technical skills, and years of experience, generate a short profile summary, andidentify candidates with at least two years of experience and Python skill.

2. Objective

To automate basic resume screening using Python Regular Expressions.

3. Algorithm

• Read the resume as plain text.
• Use Regex patterns to find email, phone number, experience, and supported technical skills. • Infer a candidate name from the first non-empty line when it is not an email or heading. • Create a short profile summary.

• Mark the candidate eligible if experience is at least two years and Python is among the detected skills.

4. Python Program

```
import re

resume = """
Alex Johnson
Email: alex.johnson@example.com Phone: +1 415-555-0182
Experienced Software Engineer with 3 years of experience.
Skills: Python, Java, SQL, Machine Learning, NLP """"

email_match = re.search(
    r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b",
    resume
)
phone_match = re.search(
    r"(?<!\w)(?:\+?\d{1,3}[-.\s]?)?(?:\(?\d{3}\)?[-.\s]?)\d{3}[-.\s]?\d{4}\b", resume
)
experience_match =
    re.search( r"(\d+(?:\.\d+)?)\s*\+?\s*years?\s+(?:of\s+)?experien ce", resume,
    re.IGNORECASE
)

lines = [line.strip() for line in resume.splitlines() if line.strip()] name = lines[0] if
lines and "@" not in lines[0] else "Not found"

skill_patterns =
    { "Python":
    r"\bPython\b", "Java":
    r"\bJava\b",
    "SQL": r"\bSQL\b",
    "Machine Learning": r"\bMachine Learning\b",
    "NLP": r"\bNLP\b"
}
skills = [
    skill for skill, pattern in skill_patterns.items()
    if re.search(pattern, resume, re.IGNORECASE)
]

email = email_match.group() if email_match else "Not found" phone =
phone_match.group() if phone_match else "Not found" years =
float(experience_match.group(1)) if experience_match else 0

summary = (
    f"{name} has {years:g} years of experience and skills in " f"{', '.join(skills) if
skills else 'no listed technical skills'}." )
eligible = years >= 2 and "Python" in skills
```

```
print("Candidate Name:", name)
print("Email:", email)
print("Mobile:", phone)
print("Technical Skills:", ", ".join(skills) if skills else "None found") print("Years
of Experience:", years)
print("Profile Summary:", summary)
print("Eligibility:", "Eligible" if eligible else "Not eligible") 5.
```

Sample Output

```
Candidate Name: Alex Johnson
Email: alex.johnson@example.com Mobile: +1 415-555-0182
Technical Skills: Python, Java, SQL, Machine Learning, NLP Years of Experience: 3
Profile Summary: Alex Johnson has 3 years of experience and skills in Python, Java, SQL, Machine Learning, NLP.
Eligibility: Eligible
```

6. Interface Illustration

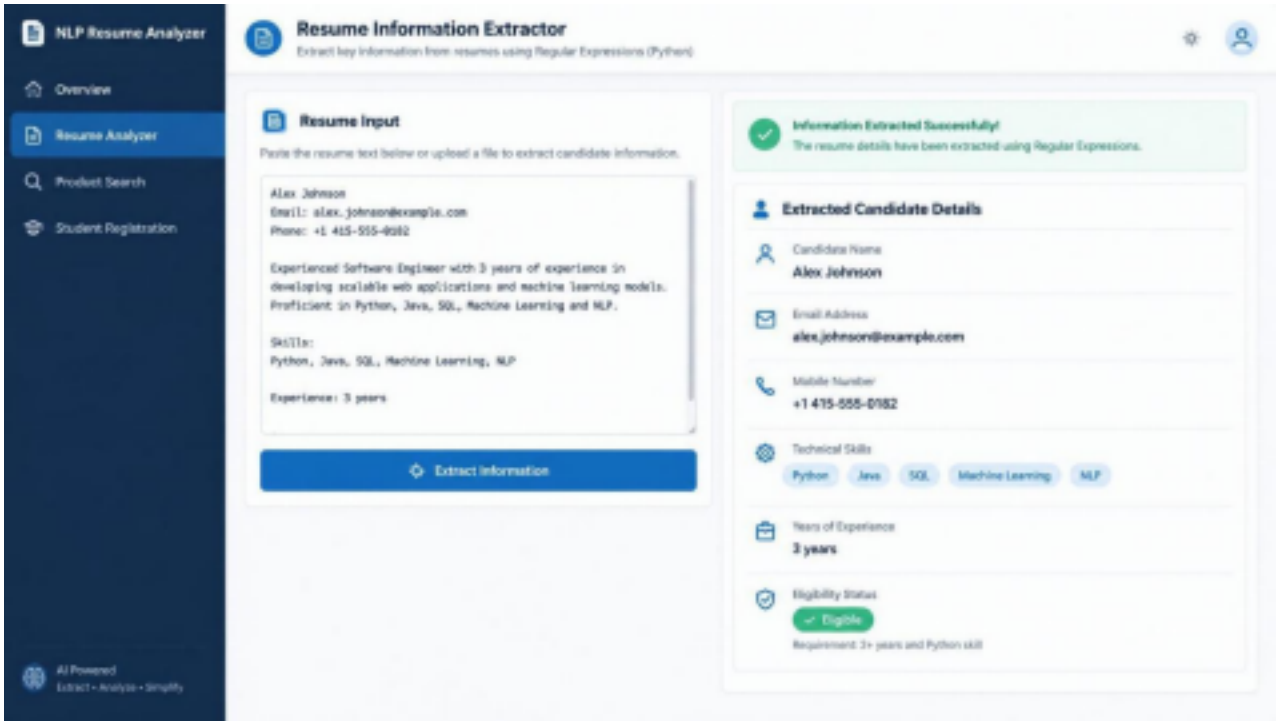


Figure 1: Illustrative Resume Information Extractor interface (mockup).

7. Result and Conclusion

The program extracts common resume fields and checks the minimum eligibility condition.
Regex-basedextraction depends on the format of the resume; a production system should handle varied layouts, international phone formats, and names more robustly.

Section 2: Product Search System

1. Problem Statement

An e-commerce catalogue needs flexible product search. The system must support exact, prefix, suffix, partial-keyword, and case-insensitive searches, display all matches, and report the number of matches.

2. Objective

To implement multiple keyword-matching modes using Regex.

3. Algorithm

• Store product names in a list.
• Read the keyword and selected search mode.
• Escape the keyword with re.escape() so special characters are treated literally. • Build the corresponding Regex pattern for exact, prefix, suffix, or partial matching. • Use

re.IGNORECASE when case-insensitive matching is selected.
• Display the matching product names and total count.

4. Python Program

```
import re

products =
    [ "Laptop Pro
    14", "Laptop
    Stand",
    "Wireless Mouse",
    "Gaming Laptop",
    "Laptop Backpack",
    "USB Keyboard",
    "Smartphone"
]

defsearch_products(keyword, mode="partial", case_insensitive=True):
    keyword = keyword.strip()
    if not keyword:
        return []

    escaped = re.escape(keyword)
    patterns = {
        "exact": rf"^{escaped}$",
        "prefix": rf"^{escaped}",
        "suffix": rf"{escaped}$",
        "partial": rf"{escaped}"
    }

    pattern = patterns.get(mode, patterns["partial"])
    flags = re.IGNORECASE if case_insensitive else 0
    return [product for product in products
            if re.search(pattern, product, flags)]

keyword = input("Enter product keyword: ")
mode = input("Choose exact/prefix/suffix/partial: ").strip().lower()
case_choice = input("Ignore case? (y/n): ").strip().lower()
ignore_case = case_choice == "y"

matches = search_products(keyword, mode, ignore_case)

print("\nMatching Products:")
if matches:
    for item in matches:
        print("-", item)
else:
    print("No matching productsfound.")

print("Total matching products:", len(matches))
```

5. Sample Runs

```
Input:
Enter product keyword: laptop
Choose exact/prefix/suffix/partial: partial
Ignore case? (y/n): y


Matching Products:
- Laptop Pro 14
- Laptop Stand
- Gaming Laptop
- Laptop Backpack
Total matching products: 4
```

Examples of search modes: exact matches the entire product name; prefix matches names starting withthekeyword; suffix matches names ending with the keyword; partial matches the keyword anywhere in the name.

6. Interface Illustration

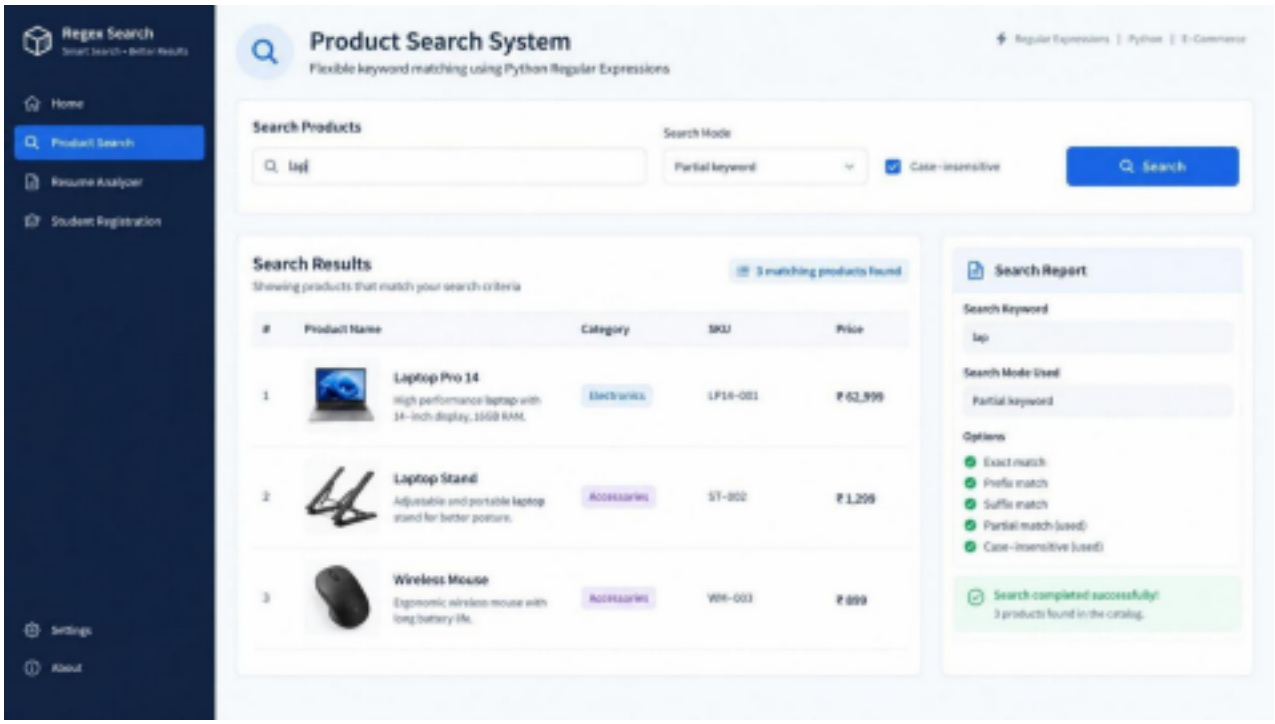


Figure 2: Illustrative Product Search System interface (mockup).

7. Result and Conclusion

The program supports four matching modes and optional case-insensitive matching. The report displays everymatching product and the total count, helping users find products flexibly.

Section 3: University Registration Validation

1. Problem Statement

A university registration portal must validate the student register number, institutional email, course code, semester, and mobile number. It must display field-wise messages and produce a final registration status.

2. Objective

To validate registration fields with Regex patterns and simple range checks.

3. Validation Rules

• Register number: SIMATS + four-digit year + one letter + three digits (example: SIMATS2026A104). • Institutional email: username followed by @saveetha.com.
• Course code: four letters followed by three digits (example: AIML301).
• Semester: integer from 1 to 8.
• Mobile number: 10 digits, beginning with 6, 7, 8, or 9.

4. Python Program

```
import re

register_no = input("Register number: ").strip()
email = input("Institutional email: ").strip()
course_code = input("Course code: ").strip().upper()
semester = input("Semester (1-8): ").strip()
mobile = input("Mobile number: ").strip()

checks = {
    "Register Number": bool(re.fullmatch(
        r"SIMATS\d{4}[A-Z]\d{3}", register_no, re.IGNORECASE
    )),
    "Institutional Email": bool(re.fullmatch(
        r"[A-Za-z0-9._%+-]+@saveetha\.com", email, re.IGNORECASE
    )),
    "Course Code":
        bool(re.fullmatch( r"[A-
```

```
        Z]{4}\d{3}", course_code
    )),
    "Semester": bool(re.fullmatch(r"[1-8]", semester)),
    "Mobile Number": bool(re.fullmatch(r"[6-9]\d{9}", mobile))
}

print("\nValidation Report")
for field, valid in checks.items():
    print(f"{field}: {'Valid' if valid else 'Invalid'}")

if all(checks.values()):
    print("Final Status: Registration Successful")
else:
    print("Final Status: Registration Failed - correct invalid fields")
```

5. Sample Run

```
Register number: SIMATS2026A104
Institutional email:student@saveetha.com
Course code: AIML301
Semester (1-8): 5
Mobile number: 9876543210

Validation Report
Register Number: Valid
Institutional Email: Valid
Course Code: Valid
Semester: Valid
Mobile Number: Valid
Final Status: Registration Successful
```

6. Interface Illustration

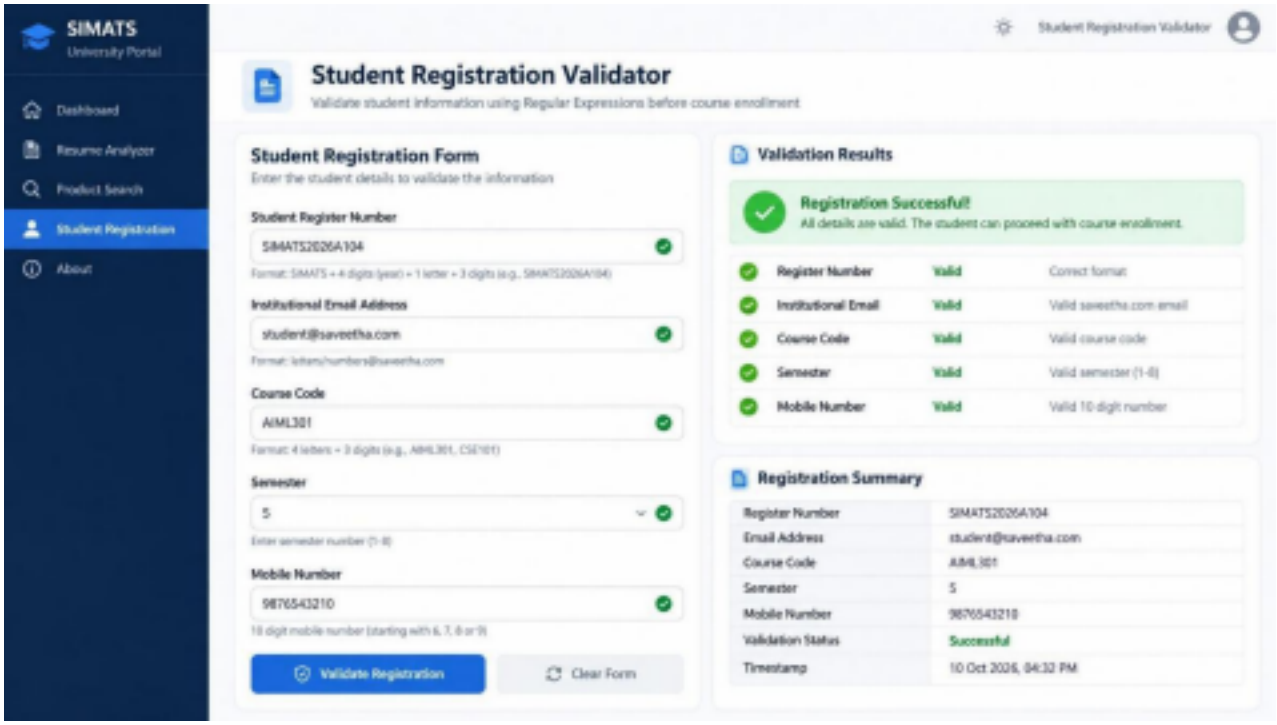


Figure 3: Illustrative Student Registration Validator interface (mockup).

7. Result and Conclusion

The program checks all five fields and reports each field's status. Registration is successful only wheneveryvalidation passes. The register-number and course-code formats are examples based on the stated assumptions; they should be adjusted to match the university's official format.

Overall Conclusion

This assessment demonstrates three practical uses of Python Regular Expressions: information extraction, flexible product searching, and input validation. Patterns, re.search(), re.fullmatch(), re.IGNORECASE, andre.escape() help automate text-processing tasks. Correctness depends on clearly defined patterns and testingagainst valid and invalid examples.

References

Python Software Foundation. “re — Regular expression operations.” https://docs.python.org/3/library/re.html Python Software Foundation. “Regular Expression HOWTO.” https://docs.python.org/3/howto/regex.html